



BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

FİNAL PROJE RAPORU

- Proje 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı
- Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması
- Proje 5 – Veri Temizleme ve ETL Süreçleri Tasarımı
- Proje 1 – Veritabanı Performans Optimizasyonu ve İzleme
- Proje 3 – Veritabanı Güvenliği ve Erişim Kontrolü

AD SOYAD: Hüseyin TINAZTEPE
NUMARA: 21290360

GITHUB:
<https://github.com/hsyntinaztepe/DB-Backup-Recovery-Automation-Prj2-Prj7>
<https://github.com/hsyntinaztepe/DB-ETL-Performance-Security-Prj5-Prj1-Prj3>

VERİTABANI: AdventureWorks2022, ETL_Proje5, Guvenlik_Proje3
TARİH: Haziran 2026

İÇİNDEKİLER

İÇİNDEKİLER.....	2
1. Proje 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı.....	4
1.1 Giriş ve Proje Amacı.....	4
1.2 Veritabanı Hazırlığı – Recovery Model.....	4
1.2.1 Recovery Model Ayarı.....	4
1.3 Yedekleme Stratejisi (Full / Differential / Log).....	4
1.3.1 Tam Yedekleme.....	4
1.3.2 Fark Yedekleme.....	4
1.3.3 Transaction Log Yedekleme.....	5
1.4 Felaket Senaryosu – Veritabanının Silinmesi.....	5
1.5 Felaketten Kurtarma – Restore İşlemi.....	5
1.5.1 Restore Zinciri.....	5
1.5.2 Restore Sonrası Kontrol.....	5
1.6 Point-in-Time Restore.....	5
1.6.1 Senaryo Hazırlığı.....	5
1.6.2 Point-in-Time Restore Uygulaması.....	6
1.7 Yedek Doğrulama.....	6
1.8 Yedekleme Geçmişi Raporlaması.....	6
1.9 SQL Sorgu Dosyaları Özeti.....	6
1.10 Sonuç ve Değerlendirme.....	7
2. Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması.....	8
2.1 Giriş ve Proje Amacı.....	8
2.2 BackupLog Tablosu Oluşturma.....	8
2.3 Manuel Tam Yedekleme.....	8
2.4 sp_AutoBackup – Otomatik Yedekleme Prosedürü.....	8
2.4.1 Prosedür Testi.....	9
2.5 SQL Server Agent Job Kurulumu.....	9
2.5.1 Job'un Manuel Çalıştırılması.....	9
2.5.2 Job Geçmişinin Kontrolü.....	9
2.6 Hata Senaryosu ve Log Kaydı.....	9
2.7 Raporlama Sorguları.....	10
2.7.1 Günlük Özet Raporu.....	10
2.7.2 Son 10 Yedek.....	10
2.7.3 msdb Yedekleme Geçmişi.....	10
2.8 Yedek Doğrulama.....	10
2.9 SQL Sorgu Dosyaları Özeti.....	10
2.10 Sonuç ve Değerlendirme.....	10
3. Proje 5 – Veri Temizleme ve ETL Süreçleri Tasarımı.....	12
3.1 Giriş ve Proje Amacı.....	12
3.2 Sistem Mimarisi (3 Katmanlı ETL).....	12
3.3 Tablo Şeması.....	12
3.4 Extract – Ham Verinin Yüklenmesi.....	12

3.5 Transform – Veri Temizleme ve Dönüştürme.....	12
3.5.1 Boşluk Temizliği ve Standartlaştırma.....	13
3.5.2 Tarih Dönüştürme ve Doğrulama.....	13
3.6 Hata Tespiti ve Load.....	13
3.7 Veri Kalitesi Raporu (Sonuçlar).....	13
3.8 SQL Sorgu Dosyaları Özeti.....	13
3.9 Sonuç ve Değerlendirme.....	14
4. Proje 1 – Veritabanı Performans Optimizasyonu ve İzleme.....	15
4.1 Giriş ve Proje Amacı.....	15
4.2 Test Ortamının Hazırlanması.....	15
4.3 Veritabanı İzleme (DMV).....	15
4.4 İndeks Yönetimi ve Performans Karşılaştırması.....	15
4.4.1 Oluşturulan İndeksler.....	15
4.4.2 Önce / Sonra Ölçümü.....	15
4.5 Sorgu İyileştirme (SARGability).....	16
4.6 Veri Yöneticisi Roller ve Erişim Yönetimi.....	16
4.7 SQL Sorgu Dosyaları Özeti.....	16
4.8 Sonuç ve Değerlendirme.....	16
5. Proje 3 – Veritabanı Güvenliği ve Erişim Kontrolü.....	17
5.1 Giriş ve Proje Amacı.....	17
5.2 Erişim Yönetimi (Authentication & Authorization).....	17
5.3 Veri Şifreleme (Encryption).....	17
5.3.1 TDE – Transparent Data Encryption.....	17
5.3.2 Kolon Bazlı Şifreleme.....	17
5.4 SQL Injection Testleri.....	17
5.5 Audit Logları.....	18
5.6 SQL Sorgu Dosyaları Özeti.....	18
5.7 Sonuç ve Değerlendirme.....	18

1. Proje 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Video Linki: <https://youtu.be/saMLoJvcOxU> (0:00 - 05:41)

1.1 Giriş ve Proje Amacı

Bu projede Microsoft tarafından yayımlanan AdventureWorks2022 örnek veritabanı kullanılmıştır. AdventureWorks, kurgusal bir bisiklet üretim şirketine ait satış, üretim, insan kaynakları ve müşteri verilerini barındıran kapsamlı bir veritabanıdır. İçeriğinde Person, Sales, Production, HumanResources ve Purchasing gibi şemalar altında 70'ten fazla tablo, çok sayıda view ve stored procedure bulunmaktadır. Yaklaşık 200 MB boyutuyla gerçek bir kurumsal veritabanını simüle etmekte olup yedekleme ve kurtarma senaryolarını test etmek için ideal bir yapı sunmaktadır.

Projenin temel amacı, kurumsal bir veritabanı için kapsamlı bir yedekleme stratejisi oluşturmak ve felaketten kurtarma (Disaster Recovery) senaryolarını uygulamalı olarak gerçekleştirmektir:

- Tam (Full), Fark (Differential) ve Log yedeklemelerinin planlanması ve uygulanması
- Kaza ile silinen veya bozulan veritabanının yedeklerden geri yüklenmesi
- Point-in-Time Restore ile belirli bir zaman noktasına geri dönme
- Yedek dosyalarının doğrulanması ve raporlanması

1.2 Veritabanı Hazırlığı – Recovery Model

1.2.1 Recovery Model Ayarı

Point-in-Time Restore ve Log yedeklemesi yapabilmek için veritabanının Recovery Model'inin FULL olması zorunludur. Aşağıdaki komutlarla bu ayar yapılmış ve doğrulanmıştır:

```
USE master;
ALTER DATABASE AdventureWorks2022 SET RECOVERY FULL;
ALTER DATABASE AdventureWorks2022 SET MULTI_USER;
SELECT name, recovery_model_desc, user_access_desc
FROM sys.databases WHERE name = 'AdventureWorks2022';
```

Sorgu sonucunda recovery_model_desc sütununda FULL değeri görülmektedir. FULL recovery model, her işlemin transaction log'a yazılmasını sağlar; bu sayede Point-in-Time Restore mümkün olur.

1.3 Yedekleme Stratejisi (Full / Differential / Log)

Proje kapsamında üç farklı yedekleme türü uygulanmıştır. Bu strateji kurumsal ortamlarda yaygın kullanılan Full + Differential + Log yedekleme zincirini oluşturmaktadır.

Yedek Türü	Dosya	Amaç	Query
Tam Yedek (Full)	AW_Full.bak	Tüm veritabanını yedekler	Query2_FullBackup.sql
Fark Yedek (Diff)	AW_Diff.bak	Son Full'dan değişenleri kaydeder	Query3_DiffBackup.sql
Log Yedek	AW_Log1/2/3.bak	Transaction log'u yedekler	Query4_LogBackup.sql

1.3.1 Tam Yedekleme

Tam yedekleme tüm veriyi ve yapıyı tek dosyaya kaydeder; kurtarma zincirinin başlangıç noktasıdır:

```
BACKUP DATABASE AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Full.bak'
WITH FORMAT, INIT, NAME = 'AW Full Backup', STATS = 10;
```

Yedek boyutu yaklaşık 200 MB olarak gerçekleşmiştir.

1.3.2 Fark Yedekleme

Fark yedeklemesi son tam yedekten bu yana değişen sayfaları kaydeder. Önce Person.Address tablosunda güncelleme yapılmış, ardından diferansiyel yedek alınmıştır:

```
UPDATE Person.Address SET ModifiedDate = GETDATE()  
WHERE AddressID BETWEEN 1 AND 100;
```

```
BACKUP DATABASE AdventureWorks2022  
TO DISK = 'C:\Backup\AdventureWorks\AW_Diff.bak'  
WITH DIFFERENTIAL, NAME = 'AW Differential Backup', STATS = 10;
```

Diferansiyel yedek boyutu yaklaşık 2 MB olup yalnızca değişen verilerin yedeklendiğini kanıtlamaktadır.

1.3.3 Transaction Log Yedekleme

Log yedeklemesi transaction günlüğünü temizleyerek Point-in-Time Restore imkânı tanır:

```
BACKUP LOG AdventureWorks2022  
TO DISK = 'C:\Backup\AdventureWorks\AW_Log1.bak'  
WITH NAME = 'AW Log Backup 1', STATS = 10;
```

1.4 Felaket Senaryosu – Veritabanının Silinmesi

Query5_Felaket.sql dosyasında gerçek bir felaket senaryosu simüle edilmiştir. Veritabanı kasıtlı olarak silinerek kurtarma sürecinin etkinliği test edilmiştir:

```
SELECT GETDATE() AS FelaketOncesiZaman;  
ALTER DATABASE AdventureWorks2022 SET SINGLE_USER WITH ROLLBACK IMMEDIATE;  
DROP DATABASE AdventureWorks2022;  
SELECT name FROM sys.databases WHERE name = 'AdventureWorks2022';
```

DROP DATABASE komutu sonrasında veritabanı tamamen silinmiştir. sys.databases sorgusunda AdventureWorks2022 artık görünmemekte; sorgu boş dönmektedir.

1.5 Felaketten Kurtarma – Restore İşlemi

1.5.1 Restore Zinciri

Silinen veritabanını geri yüklemek için Full → Differential → Log sırasıyla backup uygulanmıştır. NORECOVERY zincirin devam edeceğini; son adımdaki RECOVERY ise veritabanını açacağını belirtir:

```
RESTORE DATABASE AdventureWorks2022  
FROM DISK = 'C:\Backup\AdventureWorks\AW_Full.bak'  
WITH NORECOVERY, REPLACE, STATS = 10;  
  
RESTORE DATABASE AdventureWorks2022  
FROM DISK = 'C:\Backup\AdventureWorks\AW_Diff.bak'  
WITH NORECOVERY, STATS = 10;  
  
RESTORE LOG AdventureWorks2022  
FROM DISK = 'C:\Backup\AdventureWorks\AW_Log1.bak'  
WITH RECOVERY, STATS = 10;
```

1.5.2 Restore Sonrası Kontrol

```
SELECT name, state_desc, recovery_model_desc FROM sys.databases  
WHERE name = 'AdventureWorks2022';  
SELECT COUNT(*) AS ToplamTablo FROM INFORMATION_SCHEMA.TABLES;  
SELECT TOP 5 * FROM Person.Address;
```

Sorgu sonuçlarında state_desc = ONLINE ve tüm tabloların yerinde olduğu görülmüştür. Kurtarma işlemi başarıyla tamamlanmıştır.

1.6 Point-in-Time Restore

1.6.1 Senaryo Hazırlığı

- AW_Log2.bak alındı — bu noktaya geri dönülecek
- Person.Address tablosundaki ilk 10 kayıt kasıtlı olarak bozuldu
- Bozulma sonrası AW_Log3.bak alındı

```
BACKUP LOG AdventureWorks2022  
TO DISK = 'C:\Backup\AdventureWorks\AW_Log2.bak';
```

```
UPDATE Person.Address
SET AddressLine1 = 'BOZUK ADRES ' + CAST(AddressID AS NVARCHAR)
WHERE AddressID BETWEEN 1 AND 10;
```

```
BACKUP LOG AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Log3.bak';
```

1.6.2 Point-in-Time Restore Uygulaması

STOPAT parametresiyle bozulma öncesi zaman noktasına geri dönmüştür:

```
RESTORE DATABASE AdventureWorks2022
FROM DISK = 'AW_Full.bak' WITH NORECOVERY, REPLACE;
RESTORE DATABASE AdventureWorks2022
FROM DISK = 'AW_Diff.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log1.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log2.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log3.bak'
WITH RECOVERY, STOPAT = '2026-04-13 22:10:00';
```

Restore sonrası Person.Address tablosu kontrol edilmiş; AddressLine1 değerlerinin orijinal haline döndüğü doğrulanmıştır.

1.7 Yedek Doğrulama

RESTORE VERIFYONLY komutu ile yedeğin geçerliliği test edilir:

```
RESTORE VERIFYONLY FROM DISK = 'C:\Backup\AdventureWorks\AW_Full.bak';
```

RESTORE HEADERONLY yedek dosyasının meta verilerini listeler: yedek türü, alınma zamanı, SQL Server versiyonu. RESTORE FILELISTONLY ise yedek içindeki .mdf ve .ldf dosyalarını listeler; MOVE parametresi için kullanılır. Üç komut da başarıyla tamamlanmış; yedek dosyasının geçerli ve restore edilebilir nitelikte olduğu onaylanmıştır.

1.8 Yedekleme Geçmişi Raporlaması

```
SELECT database_name AS VeritabaniAdi,
       backup_start_date AS BaslangicZamani,
       backup_finish_date AS BitisZamani,
       DATEDIFF(SECOND, backup_start_date, backup_finish_date) AS SureSaniye,
       CASE type WHEN 'D' THEN 'Tam Yedek'
              WHEN 'I' THEN 'Fark Yedek'
              WHEN 'L' THEN 'Log Yedek' END AS YedekTuru,
       CAST(backup_size/1024/1024 AS INT) AS BoyutMB
FROM backupset WHERE database_name = 'AdventureWorks2022'
ORDER BY backup_start_date DESC;
```

1.9 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
Query1_RecoveryModel.sql	Recovery Model FULL yapma ve kontrol	Hazırlık
Query2_FullBackup.sql	Tam yedekleme alma	Full Backup
Query3_DiffBackup.sql	Veri değişikliği + fark yedekleme	Diff Backup
Query4_LogBackup.sql	Transaction log yedekleme	Log Backup
Query5_Felaket.sql	Veritabanını silme (felaket simülasyonu)	Felaket
Query6_Restore.sql	Full + Diff + Log ile geri yükleme	Kurtarma
Query7_Kontrol.sql	Restore sonrası sağlık kontrolü	Doğrulama
Query8_PointInTime.sql	Veri bozma + log yedekleme	PIT Hazırlık
Query8b_PointInTime_Restore.sql	STOPAT ile belirli zamana geri dönme	PIT Restore

Dosya Adı	İçerik	Konu
Query9_Dogrulama.sql	VERIFYONLY, HEADERONLY, FILELISTONLY	Doğrulama
Query10_Rapor.sql	Yedekleme geçmişi raporu (msdb)	Raporlama

1.10 Sonuç ve Değerlendirme

- FULL recovery model etkinleştirilerek tüm kurtarma seçeneklerine zemin hazırlanmıştır.
- Full, Differential ve Log yedeklemeleri alınarak kurumsal bir yedekleme zinciri oluşturulmuştur.
- DROP DATABASE ile felaket senaryosu simüle edilmiş ve yedeklerden tam kurtarma gerçekleştirilmiştir.
- Point-in-Time Restore ile kasıtlı bozulan veriler belirli bir zaman noktasına döndürülmüştür.
- VERIFYONLY, HEADERONLY ve FILELISTONLY ile yedek bütünlüğü doğrulanmıştır.
- msdb.dbo.backupset sorgulanarak tüm yedekleme geçmişi raporlanmıştır.

2. Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması

Video Linki: <https://youtu.be/saMLOjvcOxU> (05:42 - 10:01)

2.1 Giriş ve Proje Amacı

Bu projede de Proje 2 ile aynı AdventureWorks2022 veritabanı kullanılmıştır. AdventureWorks, Microsoft'un yayımladığı kurgusal bir bisiklet üretim şirketine ait örnek veritabanı olup satış, üretim, müşteri ve insan kaynakları verilerini barındırmaktadır. 70'ten fazla tablo ve yaklaşık 200 MB boyutuyla gerçek kurumsal ortamları simüle eden bu yapı, otomasyon ve raporlama senaryolarını test etmek için tercih edilmiştir.

- BackupLog tablosuyla yedekleme geçmişini kayıt altına almak
- sp_AutoBackup prosedürüyle Full ve Differential yedeklemeleri otomatikleştirmek
- SQL Server Agent Job ile yedeklemeyi her gece 02:00'de otomatik çalıştırmak
- Hata senaryosu oluşturup hata kayıtlarını log tablosunda gözlemlemek
- Raporlama sorguları ile yedekleme istatistiklerini analiz etmek

Not: Proje sürecinde önce SQL Server Express Edition kullanılmış; ancak SQL Server Agent'ın Express'te desteklenmediği anlaşıncaya Developer Edition'a geçilmiştir.

2.2 BackupLog Tablosu Oluşturma

Yedekleme işlemlerinin sonuçlarını kayıt altına almak için BackupLog tablosu oluşturulmuştur:

```
USE AdventureWorks2022;
GO
CREATE TABLE BackupLog (
    LogID INT IDENTITY PRIMARY KEY,
    BackupDate DATETIME DEFAULT GETDATE(),
    FilePath NVARCHAR(500),
    Status NVARCHAR(20),
    FileSizeMB FLOAT,
    DurationSec INT,
    ErrorMessage NVARCHAR(1000)
);
```

Her yedekleme işlemi tamamlandığında bu tabloya bir satır eklenmektedir. Status alanı 'BAŞARILI' veya 'HATA' değerlerinden birini alır; hata durumunda ErrorMessage sütununa açıklama yazılır.

2.3 Manuel Tam Yedekleme

Otomasyondan önce C:\Backups klasörünün erişilebilir olduğunu doğrulamak amacıyla manuel bir tam yedek alınmıştır:

```
USE AdventureWorks2022;
GO
BACKUP DATABASE AdventureWorks2022
TO DISK = 'C:\Backups\AW_Full_Manuel.bak'
WITH FORMAT, INIT, NAME = 'Manuel Tam Yedek', STATS = 10;
```

'BACKUP DATABASE successfully processed' mesajı alınmış ve AW_Full_Manuel.bak dosyasının oluştuğu gözlemlenmiştir. Dosya boyutu yaklaşık 200 MB'tır.

2.4 sp_AutoBackup – Otomatik Yedekleme Prosedürü

Hem Full hem Differential yedeklemeleri tek prosedürden yönetmek için sp_AutoBackup oluşturulmuştur. @BackupType parametresine göre davranışını değiştirir ve sonucu BackupLog'a kaydeder:

```
CREATE PROCEDURE sp_AutoBackup
    @BackupType NVARCHAR(10) = 'FULL'
AS BEGIN
    DECLARE @path NVARCHAR(500), @date NVARCHAR(50)
    DECLARE @start DATETIME = GETDATE(), @size FLOAT
    SET @date = REPLACE(REPLACE(CONVERT(NVARCHAR, GETDATE(), 120), ':', '-'), ' ', '_')
```



```

SET @path = 'C:\Backups\AW_' + @BackupType + '_' + @date + '.bak'
BEGIN TRY
    IF @BackupType = 'FULL'
        BACKUP DATABASE AdventureWorks2022 TO DISK=@path WITH FORMAT, STATS=10;
    ELSE
        BACKUP DATABASE AdventureWorks2022 TO DISK=@path WITH DIFFERENTIAL, STATS=10;
    INSERT INTO BackupLog(FilePath, Status, FileSizeMB, DurationSec)
    VALUES(@path, 'BAŞARILI', @size, DATEDIFF(SECOND,@start,GETDATE()));
END TRY
BEGIN CATCH
    INSERT INTO BackupLog(FilePath, Status, DurationSec, ErrorMsg)
    VALUES(@path, 'HATA', DATEDIFF(SECOND,@start,GETDATE()), ERROR_MESSAGE());
END CATCH
END;

```

2.4.1 Prosedür Testi

```

EXEC sp_AutoBackup @BackupType = 'FULL';
EXEC sp_AutoBackup @BackupType = 'DIFF';
SELECT * FROM BackupLog;

```

Test sonucunda C:\Backups klasöründe tarih damgalı iki .bak dosyası oluşmuş, BackupLog tablosuna 'BAŞARILI' durumuyla iki kayıt eklenmiştir. Full yedek ~200 MB, Differential yedek ~2 MB boyutundadır.

2.5 SQL Server Agent Job Kurulumu

sp_AutoBackup prosedürünün her gece 02:00'de otomatik çalışması için SQL Server Agent Job oluşturulmuştur:

```

USE msdb;
GO
EXEC sp_add_job @job_name = N'AW_Otomatik_Yedekleme';
EXEC sp_add_jobstep
    @job_name = N'AW_Otomatik_Yedekleme',
    @step_name = N'Tam Yedek Al',
    @subsystem = N'TSQL',
    @database_name = N'AdventureWorks2022',
    @command = N'EXEC sp_AutoBackup @BackupType = ''FULL''';
EXEC sp_add_schedule
    @schedule_name = N'HerGunSaat2',
    @freq_type = 4, @freq_interval = 1,
    @active_start_time = 020000;
EXEC sp_attach_schedule @job_name=N'AW_Otomatik_Yedekleme', @schedule_name=N'HerGunSaat2';
EXEC sp_add_jobserver @job_name=N'AW_Otomatik_Yedekleme';

```

2.5.1 Job'un Manuel Çalıştırılması

```

USE msdb;
EXEC sp_start_job N'AW_Otomatik_Yedekleme';

```

2.5.2 Job Geçmişinin Kontrolü

```

SELECT j.name AS JobAdi, h.run_date AS Tarih, h.run_time AS Saat,
    CASE h.run_status WHEN 0 THEN 'HATA'
    WHEN 1 THEN 'BAŞARILI' WHEN 3 THEN 'İPTAL' END AS Durum,
    h.message AS Mesaj
FROM sysjobs j JOIN sysjobhistory h ON j.job_id = h.job_id
WHERE j.name = 'AW_Otomatik_Yedekleme'
ORDER BY h.run_date DESC, h.run_time DESC;

```

Sorgu sonucunda Job'un BAŞARILI durumuyla çalıştığı görülmüştür. 'The job succeeded. The job was invoked by User' mesajı başarılı çalışmanın kanıtıdır.

2.6 Hata Senaryosu ve Log Kaydı

Prosedürün hata durumundaki davranışını test etmek için C:\Backups klasörü C:\Backups_old olarak yeniden adlandırılmış ve sp_AutoBackup çalıştırılmıştır. SQL Server hedef klasörü bulamadığından yedekleme başarısız olmuştur. Hata mesajı: 'BACKUP DATABASE is terminating abnormally.'

BackupLog tablosunda Status = 'HATA' ve ErrorMsg sütununda hata açıklaması gözlemlenmiştir. Klasör geri adlandırılıp prosedür tekrar çalıştırıldığında 'BAŞARILI' kaydı düşmüştür.

LogID	Durum	FileSizeMB	Açıklama
1, 3, 6–9	BAŞARILI	~200 MB / ~2 MB	Normal yedeklemeler
5	HATA	NULL	Klasör bulunamadı – terminating abnormally

2.7 Raporlama Sorguları

2.7.1 Günlük Özet Raporu

```
SELECT CONVERT(DATE, BackupDate) AS Tarih,  
COUNT(*) AS ToplamYedek,  
SUM(CASE WHEN Status='BAŞARILI' THEN 1 ELSE 0 END) AS Basarili,  
SUM(CASE WHEN Status='HATA' THEN 1 ELSE 0 END) AS Hatali,  
AVG(FileSizeMB) AS OrtBoyutMB, AVG(DurationSec) AS OrtSureSaniye  
FROM BackupLog  
GROUP BY CONVERT(DATE, BackupDate) ORDER BY Tarih DESC;
```

2.7.2 Son 10 Yedek

```
SELECT TOP 10 * FROM BackupLog ORDER BY BackupDate DESC;
```

2.7.3 msdb Yedekleme Geçmişi

```
SELECT TOP 10 database_name, backup_start_date, backup_finish_date,  
backup_size/1024/1024 AS SizeMB,  
CASE type WHEN 'D' THEN 'Tam Yedek'  
WHEN 'I' THEN 'Diferansiyel'  
WHEN 'L' THEN 'Log Yedeği' END AS YedekTipi  
FROM msdb.dbo.backupset WHERE database_name = 'AdventureWorks2022'  
ORDER BY backup_finish_date DESC;
```

2.8 Yedek Doğrulama

```
RESTORE VERIFYONLY  
FROM DISK = 'C:\Backups\AW_Full_Manuel.bak';
```

Komut 'The backup set on file 1 is valid.' mesajıyla tamamlanmıştır. Yedek dosyasının bütün, okunabilir ve restore edilebilir nitelikte olduğu garanti edilmiştir.

2.9 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
Query1_BackupLog_Tablo.sql	BackupLog tablosunu oluşturma	Tablo
Query2_Manuel_FullBackup.sql	Manuel tam yedek alma	Full Backup
Query3_AutoBackup_Procedure.sql	sp_AutoBackup prosedürü	Otomasyon
Query4_Procedure_Test.sql	Full ve Diff prosedür testi	Test
Query5_AgentJob_Olustur.sql	Agent Job ve zamanlama kurulumu	Agent Job
Query6_AgentJob_Calistir.sql	Job'u manuel tetikleme	Çalıştırma
Query7_Job_Gecmis_Kontrol.sql	Job geçmişi sorgulama	Kontrol
Query8_Raporlama.sql	Günlük özet, son 10, msdb raporu	Raporlama
Query9_Dogrulama.sql	RESTORE VERIFYONLY	Doğrulama
Query10_BackupLog_Goruntule.sql	BackupLog tablosunu görüntüleme	Görüntüleme
Query11_VerifyOnly_Dogrula.sql	Son yedek dosyasını doğrulama	Doğrulama

2.10 Sonuç ve Değerlendirme

- BackupLog tablosu ile tüm yedekleme işlemlerinin başarı/hata durumu kayıt altına alınmıştır.

- sp_AutoBackup prosedürü sayesinde Full ve Differential yedeklemeler tek komutla yönetilebilir hale gelmiştir.
- SQL Server Agent Job ile yedekleme günlük 02:00'de otomatik çalışacak şekilde zamanlanmıştır.
- Hata senaryosunda prosedürün hatayı yakalayıp log tablosuna kaydettiği doğrulanmıştır.
- Günlük özet ve msdb geçmiş sorguları ile yedekleme istatistikleri raporlanmıştır.
- RESTORE VERIFYONLY komutuyla yedek dosyalarının geçerliliği teyit edilmiştir.

3. Proje 5 – Veri Temizleme ve ETL Süreçleri Tasarımı

Video Linki: <https://youtu.be/6GHcj0e-f3w> (00:00 - 10:40)

3.1 Giriş ve Proje Amacı

Bu projede, gerçek hayattaki kirli verileri temsil eden 1050 satırlık bir müşteri veri seti üzerinde ETL (Extract, Transform, Load) süreci uygulanmıştır. Çalışma için ETL_Proje5 adında ayrı bir veritabanı oluşturulmuştur. ETL; veriyi kaynaktan çekme (Extract), iş kurallarına göre temizleyip dönüştürme (Transform) ve hedef veritabanına yükleme (Load) sürecidir ve veri ambarı uygulamalarının temelini oluşturur.

Projenin amacı, kasıtlı olarak bozukluk içeren bir veri setini SQL ile temizlemek, standartlaştırmak, hedef tabloya yüklemek ve sürecin sonunda bir veri kalitesi raporu üretmektir:

- Hatalı, eksik ve tutarsız verilerin SQL ile temizlenmesi
- Farklı formatlardaki verilerin standartlaştırılması ve dönüştürülmesi
- Geçerli verilerin doğru hedef tabloya yüklenmesi, reddedilenlerin loglanması
- Veri temizleme sürecine dair kalite raporlarının oluşturulması

3.2 Sistem Mimarisi (3 Katmanlı ETL)

Süreç, profesyonel ETL uygulamalarında kullanılan üç katmanlı bir yapı üzerine kurulmuştur. Staging katmanı ham veriyi (tüm kolonlar NVARCHAR) hiç değiştirmeden tutar; Clean katmanı tüm kurallardan geçen kayıtları doğru tiplerle saklar; Error Log katmanı ise reddedilen kayıtları sebebiyle birlikte kaydeder.

Katman	Tablo	Açıklama
Staging	staging_customers	Ham veri, tüm kolonlar NVARCHAR
Clean	clean_customers	Temizlenmiş veri, doğru tipler (INT, DATE)
Error	etl_error_log	Reddedilen kayıtlar ve hata sebepleri

3.3 Tablo Şeması

Üç katman için oluşturulan tablolar aşağıdaki gibidir:

```
CREATE TABLE staging_customers (  
  id NVARCHAR(50), full_name NVARCHAR(200),  
  email NVARCHAR(200), city NVARCHAR(100),  
  signup_date NVARCHAR(50), phone NVARCHAR(50));  
  
CREATE TABLE clean_customers (  
  id INT PRIMARY KEY, full_name NVARCHAR(200) NOT NULL,  
  email NVARCHAR(200) NOT NULL, city NVARCHAR(100),  
  signup_date DATE, phone NVARCHAR(20),  
  loaded_at DATETIME DEFAULT GETDATE());  
  
CREATE TABLE etl_error_log (  
  error_id INT IDENTITY(1,1) PRIMARY KEY,  
  source_id NVARCHAR(50), error_type NVARCHAR(100),  
  error_detail NVARCHAR(500), logged_at DATETIME DEFAULT GETDATE());
```

3.4 Extract – Ham Verinin Yüklenmesi

Ham CSV dosyası BULK INSERT ile staging tablosuna olduğu gibi aktarılmıştır. Bu aşamada hiçbir temizlik yapılmaz. CSV UTF-8 BOM kodlamasında olduğu için Türkçe karakterlerin korunması adına CODEPAGE 65001 kullanılmıştır.

```
BULK INSERT dbo.staging_customers  
FROM 'C:\etl\customers_raw.csv'  
WITH (FIRSTROW = 2, FIELDTERMINATOR = ',',  
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
```

3.5 Transform – Veri Temizleme ve Dönüştürme

3.5.1 Boşluk Temizliği ve Standartlaştırma

İsim, e-posta ve şehir alanlarındaki baş/son boşluklar LTRIM ve RTRIM ile temizlenmiştir. Şehir adları farklı yazımlarda geldiği için (istanbul, ISTANBUL, " İstanbul ") tümü UPPER ile büyük harfe çevrilerek standartlaştırılmış, e-postalar küçük harfe indirgenmiştir.

```
UPDATE staging_customers
SET full_name = LTRIM(RTRIM(full_name)),
    email = LTRIM(RTRIM(email)), city = LTRIM(RTRIM(city));
UPDATE staging_customers SET city = UPPER(city);
UPDATE staging_customers SET email = LOWER(email);
```

3.5.2 Tarih Dönüştürme ve Doğrulama

Tarihler gg.aa.yyyy formatında tutulmaktadır (MSSQL format kodu 104). TRY_CONVERT fonksiyonu dönüştürülemeyen değerler için hata fırlatmak yerine NULL döndürür; bu sayede bozuk tarihler kolayca tespit edilir.

```
TRY_CONVERT(DATE, signup_date, 104) -- başarısızsa NULL döner
```

3.6 Hata Tespiti ve Load

Her temizlik kuralı için kuralı geçemeyen kayıtlar etl_error_log tablosuna sebebiyle birlikte yazılır. Ardından yalnızca tüm kuralları geçen kayıtlar clean_customers tablosuna yüklenir. Tekrar eden e-postalarda ROW_NUMBER / NOT EXISTS mantığı ile yalnızca ilk kayıt kabul edilir.

```
INSERT INTO clean_customers (id, full_name, email, city, signup_date, phone)
SELECT TRY_CONVERT(INT, id), full_name, email, city,
    TRY_CONVERT(DATE, signup_date, 104), phone
FROM staging_customers s
WHERE TRY_CONVERT(INT, id) IS NOT NULL
    AND full_name <> '' AND email LIKE '%@%._%'
    AND TRY_CONVERT(DATE, signup_date, 104) IS NOT NULL
    AND NOT EXISTS ( ... ); -- duplicate email kontrolü
```

3.7 Veri Kalitesi Raporu (Sonuçlar)

ETL süreci tamamlandıktan sonra üretilen genel özet aşağıdaki gibidir:

Metrik	Değer
Toplam ham kayıt	1050
Temiz tabloya yüklenen	759
Reddedilen kayıt	291
Başarı oranı	%72.29

Reddedilen kayıtların hata tipine göre dağılımı:

Hata Tipi	Adet	Açıklama
INVALID_EMAIL	104	Geçersiz e-posta deseni
INVALID_DATE	94	Tarih gg.aa.yyyy formatına çevreilemedi
DUPLICATE_EMAIL	48	Tekrar eden e-posta (ilki dışında)
NULL_NAME	42	İsim alanı boş
INVALID_ID	32	Kimlik değeri sayısal değil

Temizlik sonrası şehir dağılımında 7 standart şehir adı (hepsi büyük harf, boşluksuz) elde edilmiş; doğrulama sorgusunda temiz tabloda hiç bozuk e-posta, null tarih veya duplicate kalmadığı (tümü sıfır) görülmüştür.

3.8 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
01_create_tables.sql	Veritabanı ve 3 katmanlı tablolar	Hazırlık

Dosya Adı	İçerik	Konu
02_extract.sql	CSV'yi staging'e yükleme (BULK INSERT)	Extract
03_transform_load.sql	Temizlik, standartlaştırma, yükleme, loglama	Transform/Load
04_quality_report.sql	Veri kalitesi raporu sorguları	Raporlama
00_run_all.sql	Tüm adımları sırayla çalıştıran master script	Otomasyon

3.9 Sonuç ve Değerlendirme

- Üç katmanlı ETL mimarisi (staging / clean / error) ile süreç hatalara dayanıklı ve denetlenebilir kılınmıştır.
- TRY_CONVERT kullanılarak bozuk verinin süreci durdurması engellenmiştir.
- 1050 ham kayıttan %72.29'u temizlenerek hedef tabloya yüklenmiştir.
- Reddedilen 291 kayıt sebepleriyle loglanarak izlenebilir hale getirilmiştir.
- Veri kalitesi raporu ile sürecin başarısı somut sayılarla ortaya konmuştur.

4. Proje 1 – Veritabanı Performans Optimizasyonu ve İzleme

Video Linki: <https://youtu.be/6GHcj0e-f3w> (10:41 - 17:49)

4.1 Giriş ve Proje Amacı

Bu projede büyük bir veritabanı tablosu üzerinde performans analizi yapılmış; indeksleme, sorgu iyileştirme ve izleme teknikleriyle sorgu süreleri ölçülebilir biçimde düşürülmüştür. Çalışma AdventureWorks2022 veritabanı içinde üretilen 500.000 satırlık SalesOrdersBig tablosu üzerinde yürütülmüştür.

- DMV'lerle sorgu performansının ve eksik indekslerin izlenmesi
- Doğru indekslerin oluşturulması, gereksiz indekslerin kaldırılması
- Aynı sorgunun indeksli/indekssiz performansının karşılaştırılması
- Farklı kullanıcı rolleri için erişim yönetimi

4.2 Test Ortamının Hazırlanması

Performans farkını ölçebilmek için yeterince büyük veri gereklidir. AdventureWorks2022 içinde 500.000 satırlık SalesOrdersBig tablosu üretilmiştir. Tablo bilinçli olarak indeksiz oluşturulmuş; böylece optimizasyon öncesi kötü performans ölçülebilmıştır. Veri ROW_NUMBER ve CHECKSUM(NEWID()) ile rastgele üretilmiştir.

```
CREATE TABLE dbo.SalesOrdersBig (  
    OrderID INT, CustomerID INT, ProductID INT,  
    OrderDate DATETIME, Quantity INT, UnitPrice DECIMAL(10,2),  
    Region NVARCHAR(50), Status NVARCHAR(20));  
-- 500.000 satır CTE Numbers yaklaşımıyla üretildi
```

4.3 Veritabanı İzleme (DMV)

SQL Profiler yerine, daha hafif ve modern olan Dynamic Management View'lar kullanılmıştır. Bu görünümeler sistemin canlı performans verisini sunar ve optimizasyon kararlarının tahminle değil ölçümle alınmasını sağlar.

- sys.dm_exec_query_stats: En çok CPU ve süre tüketen sorguları listeler.
- sys.dm_os_wait_stats: Sistemin nerede beklediğini (disk, CPU, kilit) gösterir.
- sys.dm_db_missing_index_details: SQL Server'ın önerdiği eksik indeksleri verir.
- sys.dm_db_index_usage_stats: Hangi indeksin kullanıldığını/kullanılmadığını gösterir.

4.4 İndeks Yönetimi ve Performans Karşılaştırması

Projenin merkezi bölümüdür. Aynı sorgular önce indeksiz, sonra indeksli çalıştırılarak fark ölçülmüştür. Ölçüm için SET STATISTICS IO (mantıksal okuma) ve Actual Execution Plan (Table Scan vs Index Seek) kullanılmıştır.

4.4.1 Oluşturulan İndeksler

- Clustered index (OrderID): Tablonun fiziksel sıralamasını belirler.
- Covering nonclustered index (CustomerID + INCLUDE): Müşteri bazlı sorguları tek indeksten karşılar.
- Composite index (Region, Status, OrderDate): Çok kolonlu filtre + sıralama sorgularını hızlandırır.

4.4.2 Önce / Sonra Ölçümü

CustomerID filtreli sorgu için indeks öncesi ve sonrası sonuçlar aşağıdaki gibidir:

Durum	Çalıştırma Planı	Sonuç
İndeks YOK	Table Scan	Tüm 500.000 satır taranır, yüksek okuma
İndeks VAR	Index Seek	Doğrudan ilgili satırlara gidilir, düşük okuma

İndeks eklendikten sonra sorgunun çalıştırma planı Table Scan'den Index Seek'e geçmiş, mantıksal okuma sayısı dramatik biçimde düşmüştür.

4.5 Sorgu İyileştirme (SARGability)

Aynı sonucu veren iki sorgu, yazım şekline göre çok farklı performans gösterebilir. Filtre kolonuna fonksiyon uygulamak mevcut indeksi devre dışı bırakır ve tablo taramasına yol açar.

```
-- KÖTÜ: YEAR() fonksiyonu indeksi devre dışı bırakır (Scan)
WHERE YEAR(OrderDate) = 2024

-- İYİ: aralık koşulu indeksi kullanabilir (Seek)
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'
```

4.6 Veri Yöneticisi Roller ve Erişim Yönetimi

Performans yönetimi aynı zamanda bir yetki konusudur: kim sorgu çalıştırabilir, kim indeks değiştirebilir? Bu kapsamda iki rol tanımlanmıştır.

Kullanıcı	Roller	Yetkiler
rapor_kullanici	db_datareader	Sadece SELECT; veri değiştirme DENY
veri_yonetici	db_datareader + db_datawriter	Okuma + yazma + ALTER (indeks)

4.7 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
01_create_big_table.sql	500.000 satırlık test tablosu	Hazırlık
02_monitoring_dmv.sql	DMV ile izleme sorguları	İzleme
03_index_optimization.sql	İndeksleme + önce/sonra ölçüm	Optimizasyon
04_roles_access.sql	Kullanıcı rolleri ve yetkiler	Erişim

4.8 Sonuç ve Değerlendirme

- 500.000 satırlık tabloda performans sorunları DMV'lerle tespit edilmiştir.
- Doğru indekslerle sorgular Table Scan yerine Index Seek kullanmış, okuma sayısı dramatik düşmüştür.
- SARGability ilkesiyle sorgu yazımının performansa etkisi gösterilmiştir.
- Rol tabanlı erişim yönetimiyle performansa müdahale yetkisi kontrol altına alınmıştır.

5. Proje 3 – Veritabanı Güvenliği ve Erişim Kontrolü

Video Linki: <https://youtu.be/6GHcj0e-f3w> (17:51 - 25:52)

5.1 Giriş ve Proje Amacı

Bu projede hassas veri içeren bir veritabanı dört farklı güvenlik ekseninde korunmuştur. Çalışma için Guvenlik_Proje3 veritabanı ve içinde TC kimlik, maaş gibi hassas alanlar barındıran Calisanlar tablosu oluşturulmuştur.

- Erişim yönetimi: SQL Server Authentication, rol ve kolon bazlı yetkilendirme
- Veri şifreleme: TDE (tüm veritabanı) ve kolon bazlı şifreleme
- SQL injection açığının gösterimi ve parametrelili sorgu ile engellenmesi
- Audit logları ile kullanıcı aktivitelerinin izlenmesi

5.2 Erişim Yönetimi (Authentication & Authorization)

SQL Server'da güvenlik iki katmanlıdır: kimlik doğrulama (SQL Server veya Windows Authentication) ve yetkilendirme (rol/izin). Bu projede iki kullanıcı tanımlanmış; rapor_kullanici'nin hassas kolonları (Maas, TCKimlik) görmesi DENY ile engellenmiştir.

```
GRANT SELECT ON dbo.Calisanlar TO rapor_kullanici;  
DENY SELECT ON dbo.Calisanlar (Maas, TCKimlik) TO rapor_kullanici;
```

EXECUTE AS ile rapor_kullanici kimliğine geçilip SELECT * denendiğinde hassas kolonlar yüzünden erişim reddedilmiş; yalnızca izinli kolonlar seçilebilmiştir. Bu, kısıtlamanın çalıştığının kanıtıdır.

5.3 Veri Şifreleme (Encryption)

5.3.1 TDE – Transparent Data Encryption

TDE, veritabanının disk üzerindeki dosyalarını AES-256 ile şifreler; dosyalar çalınsa bile anahtar olmadan okunamaz. Master Key → Sertifika → Database Encryption Key hiyerarşisi kullanılır. Şifreleme durumu sys.dm_database_encryption_keys ile kontrol edilmiş, durum kodu 3 (tam şifreli) görülmüştür.

```
CREATE DATABASE ENCRYPTION KEY  
WITH ALGORITHM = AES_256  
ENCRYPTION BY SERVER CERTIFICATE TDE_Sertifika;  
ALTER DATABASE Guvenlik_Proje3 SET ENCRYPTION ON;
```

5.3.2 Kolon Bazlı Şifreleme

TC kimlik numaraları, simetrik anahtar ve ENCRYPTBYKEY ile şifrelenerek VARBINARY bir kolonda saklanmıştır. Anahtar açılmadan kolon okunamaz; açıldığında DECRYPTBYKEY ile çözülür.

```
OPEN SYMMETRIC KEY KolonAnahtar DECRYPTION BY CERTIFICATE KolonSertifika;  
UPDATE dbo.Calisanlar  
SET TCKimlik_Sifreli = ENCRYPTBYKEY(KEY_GUID('KolonAnahtar'), TCKimlik);  
CLOSE SYMMETRIC KEY KolonAnahtar;
```

5.4 SQL Injection Testleri

SQL injection, kullanıcı girdisinin sorgu metnine doğrudan yapıştırılmasıyla oluşan en yaygın güvenlik açığıdır. String birleştirmeli sorguda saldırgan ' OR '1'='1' girdisiyle WHERE koşulunu her zaman doğru yapar ve tüm tabloyu ele geçirir.

```
-- Saldırı girdisi: ' OR '1'='1'  
-- Oluşan sorgu: WHERE Email = '' OR '1'='1'  
-- '1'='1' her zaman DOĞRU → bütün kayıtlar döner!
```

Çözüm, girdiyi parametre olarak geçirmektir; böylece girdi 'kod' değil 'veri' olarak işlenir ve saldırı etkisiz kalır.

```
EXEC sp_executesql
    N'SELECT ... WHERE Email = @p_email',
    N'@p_email NVARCHAR(100)', @p_email = @email;
-- Saldırı girdisi artık sadece aranan metin; sonuç boş döner.
```

5.5 Audit Logları

Güvenliğin son halkası izlenebilirliktir. SQL Server Audit ile Calisanlar tablosu üzerindeki SELECT, INSERT, UPDATE ve DELETE işlemleri loglanmıştır. Loglar sys.fn_get_audit_file ile okunarak hangi kullanıcının ne zaman hangi sorguyu çalıştırdığı görülebilir.

Loglanan Bilgi	Açıklama
event_time	İşlemin gerçekleştiği zaman
server_principal_name	İşlemi yapan kullanıcı
action_id	İşlem türü (SELECT/UPDATE vb.)
statement	Çalıştırılan sorgu metni

5.6 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
01_setup_database.sql	Veritabanı ve hassas veri tablosu	Hazırlık
02_access_control.sql	Login, kullanıcı, rol ve yetkiler	Erişim
03_encryption.sql	TDE + kolon bazlı şifreleme	Şifreleme
04_sql_injection.sql	Injection açığı + parametrelili korunma	Injection
05_audit_logging.sql	SQL Server Audit ile loglama	Audit

5.7 Sonuç ve Değerlendirme

- Erişim yönetimiyle kullanıcıların yalnızca yetkili oldukları veriye erişmesi sağlanmıştır.
- TDE ve kolon şifrelemesiyle veri hem diskte hem veritabanı içinde okunamaz hale getirilmiştir.
- SQL injection açığı gösterilmiş ve parametrelili sorgu ile kapatılmıştır.
- Audit loglarıyla tüm erişimler izlenebilir kılınmıştır.
- Katmanlı yaklaşım (defense in depth) ile birden fazla koruma katmanı üst üste bindirilmiştir.